

Optimizing Automatic Differentiation in Julia: Comparison with Pytorch and Flux

1st Rafał Pyłko

Department of Electrical Engineering
Warsaw University of Technology
Warsaw, Poland
rafal.pylko.stud@pw.edu.pl

2nd Michał Jagodziński

Department of Electrical Engineering
Warsaw University of Technology
Warsaw, Poland
michal.jagodzinski2.stud@pw.edu.pl

Abstract—This article presents a custom AD engine built from scratch in Julia, implementing backward accumulation via a computational graph. To achieve competitive performance, we applied several low-level optimizations: im2col transformations for convolutional layers, strict type stability enforcement, Float32 precision, and memory-efficient in-place operations combined with loop vectorization (@simd) and bounds checking elimination (@inbounds). We benchmarked our optimized library against PyTorch and Flux on a standard CNN architecture with full FashionMNIST dataset. Results show that our optimized implementation completes three training epochs in about 19 seconds, compared to 30 seconds for Flux and 33 seconds for PyTorch. Additionally, our engine’s memory allocation is only about 10% of that required by Flux. This study demonstrates that Julia can deliver exceptional performance for custom deep learning tools.

Index Terms—Julia, CNN, optimization, PyTorch, Neural Networks, SDG, Auto Differentiation

I. INTRODUCTION

Deep learning architectures, particularly Convolutional Neural Networks (CNNs), demand massive computational resources and highly efficient gradient computation. Automatic Differentiation (AD) serves as the mathematical foundation of this process, enabling the dynamic and accurate computation of derivatives necessary for backpropagation. While established frameworks like PyTorch and Flux provide highly optimized, production-ready AD engines, their heavily abstracted nature often obscures the underlying performance bottlenecks and memory management challenges.

The Julia programming language presents a compelling environment for deep learning by solving the “two-language problem” – promising high-level, mathematically intuitive syntax combined with C-like execution speed. However, because Julia is dynamically typed, realizing its full performance potential requires deliberate architectural decisions. Naive implementations of neural network training in Julia frequently suffer from type instability, excessive memory allocation, and unoptimized loops.

To investigate and overcome these fundamental bottlenecks, we developed a custom AD library from scratch, implementing backward accumulation over a dynamic computational graph. Building a custom engine provides control over the entire execution pipeline. This article explores the critical optimizations required to make a custom Julia AD engine highly

performant. By applying techniques such as im2col transformations, Float32 precision enforcement, strict type stability, and memory-efficient in-place operations, we demonstrate how to effectively bypass the overhead.

II. LITERATURE REVIEW

Automatic Differentiation has become a fundamental tool in modern machine learning and numerical optimization. As Baydin et al. [1] highlight in their comprehensive survey, AD is strictly distinct from both numerical approximation and symbolic differentiation. Instead of estimating derivatives or generating massive mathematical expressions, AD evaluates exact derivatives by breaking down computer programs into elementary operations. For machine learning models, which typically have thousands or millions of parameters and a single scalar loss output, the reverse mode of AD is the most efficient and widely used approach.

Most popular deep learning frameworks, such as PyTorch [4], rely on dynamic computational graphs to implement reverse-mode AD. During the forward pass, PyTorch dynamically records the executed operations to build a graph, which is then traversed backward to compute gradients. While this method is highly flexible and user-friendly, it introduces runtime overhead. Furthermore, tools like PyTorch face the “two-language problem”: they provide a convenient Python interface, but for performance reasons, the actual AD engine and mathematical operations must be written in low-level languages like C++.

The Julia programming language [6] was designed specifically to solve this two-language problem. Thanks to its Just-In-Time (JIT) compilation and multiple dispatch system, Julia allows developers to write high-level, readable code that runs at speeds comparable to C. This makes Julia an excellent environment for developing AD systems. Early solutions, such as the ForwardDiff package [2], successfully utilized Julia’s type system to implement forward-mode AD using dual numbers and operator overloading.

Beyond runtime dynamic graphs, optimizing reverse-mode AD requires integrating the differentiation process directly into the compilation pipeline. One such method is the “source-to-source” approach, utilized by the Zygote framework [5]. By transforming the program’s intermediate representation (SSA

form) during compilation, Zygote effectively eliminates the runtime overhead typical of graph tracing. Furthermore, AD optimizations can be pushed even deeper, closer to the hardware itself. As demonstrated by Moses and Churavy [7] with the Enzyme tool, performing automatic differentiation directly at the LLVM compiler level unlocks low-level performance enhancements well before the final machine code is generated.

Finally, while an efficient AD engine is crucial, the overall performance of a deep learning library heavily depends on the underlying mathematical operations. For example, CNNs require massive amounts of computation. Implementing convolutions using naive nested loops is highly inefficient. As demonstrated by Chellapilla et al. [3], this bottleneck can be overcome by unrolling input images into 2D matrices – a technique known as *im2col*. By reshaping the data, the complex convolution operation is transformed into a highly optimized General Matrix Multiplication.

III. IMPLEMENTATION AND OPTIMIZATION

A. Graph Architecture and AD Engine

The custom automatic differentiation library operating in reverse mode was implemented based on a directed acyclic computational graph. The intermediate nodes represent mathematical operations, while the leaves correspond to either input data tensors or trainable model parameters.

The graph topology is built dynamically during the network initialization phase. The derivative calculation is performed in two stages. First, a topological sorting algorithm based on depth-first search (DFS) generates a linear execution sequence of the nodes. Subsequently the forward propagation process is executed, followed by the backward propagation of gradients which is executed by sequentially calling the adjoint operators in reverse topological order.

The whole AD engine is based on the parametrized node structure: `GraphNode{OP, N}`, where `OP` is a symbol representing the operation (e.g. `:relu`), and `N` specifies the number of parent nodes. This design leverages Julia’s multiple dispatch mechanism, allowing the Just-In-Time (JIT) compiler to generate specialized, highly optimized machine code independently for each operation type.

Every operator defines two fundamental functions:

- `primal!(y::GraphNode{OP, N})` – Responsible for the forward pass phase. It computes the node’s value based on the data contained in its input arguments.
- `adjoint!(y::GraphNode{OP, N})` – Implements the backward pass step. It computes the partial derivatives and accumulates gradients in the input nodes.

B. Implementation Challenges

The primary scientific and engineering challenge during the library’s development was ensuring the mathematical correctness and numerical stability of adjoint operators for multidimensional operations, alongside efficient memory management.

Numerical stability and avoiding Jacobian allocation: In the SoftMax layer, the standard approach to computing

the gradient would require allocating a full, square Jacobian matrix, which would drastically increase memory complexity. This challenge was resolved by directly hardcoding the product of the Jacobian matrix and the upstream gradient vector using the following vector identity:

$$x.\text{grad} += y.\text{data} \odot (y.\text{grad} - \sum (y.\text{grad} \odot y.\text{data}))$$

This was implemented directly within the layer’s backward step. Additionally, to prevent numerical overflow in the forward pass (`primal!`), the input values were translated by subtracting their local maximum. Similarly, in the CrossEntropy loss function, a small constant $\epsilon = 10^{-10}$ was introduced to prevent computing the logarithm of zero.

Stateless network operators: Classic deep learning frameworks often suffer from a lack of separation between a layer’s definition and its execution state. Our implementation adopts a functional programming paradigm: the operator structures themselves (such as `MaxPool` or `ConvLayer`) are immutable and stateless. The entire dynamic computational state—including output tensors, gradients, and cached data like dropout masks or pooling indices—is allocated and bound to a specific `GraphNode` instance only when the graph is built. This design allows for safe weight sharing and easy reuse of the same operators.

Sequential batch processing: Instead of implementing full 4D tensor processing, which would require massive memory buffers, we opted for an architectural trade-off. The `train_step!` procedure processes data batches sequentially using memory-safe `@view` macros. As a result, the memory required for convolutional transformations is allocated exclusively for a single sample. This drastically reduces RAM consumption at the cost of a minimal overhead in computational throughput.

C. Applied Performance Optimizations

To achieve execution speeds comparable to low-level languages like C or Fortran, several algorithmic optimizations were implemented, taking advantage of Julia’s unique JIT compiler mechanisms.

The im2col algorithm and GEMM reduction: A naive implementation of 2D convolutions using nested loops fails to utilize the CPU cache effectively. In our `ConvLayer`, we implemented the `im2col!` transformation, which reshapes the 3D input tensor into a 2D matrix. Consequently, both the forward convolution and its adjoint operation are reduced to General Matrix Multiplication (GEMM). These operations are executed in-place using the highly optimized LinearAlgebra (BLAS) library via the `mul!` function.

In-place operations and loop fusion: To reduce the workload on the Garbage Collector, we always used vectorized operations using the `@.` macro and the dot-assignment operator (`.=`). Julia automatically performs loop fusion on these expressions, executing the entire operation in a single loop without allocating temporary arrays. Furthermore, the operational buffers for the `im2col` transformation are pre-allocated only once during the layer’s initialization.

Type stability annotations: Using dictionaries to represent cache memory (`Dict{Symbol, Any}`) inherently introduces type instability in Julia. This challenge was bypassed by applying explicit type assertions to the variables extracted from the dictionaries (e.g., asserting `:Matrix{Float32}` or `:Int`) immediately before they are used in loops. This practice allows the compiler to generate optimal machine code without the overhead of dynamic type checking.

Low-level compiler directives: In the most performance-critical sections of the code responsible for matrix indexing (such as the `im2col!`, `col2im!`, and `MaxPool` functions), the loops were annotated with the `@inbounds` and `@simd` macros. The former disables costly array bounds checking, while the latter forces the compiler to vectorize loop instructions (utilizing AVX processor registers). This drastically improves the execution speed for operations on contiguous memory elements arranged in column-major order.

IV. TEST AND COMPARISON

To evaluate our engine, we compared its performance against two widely used libraries: Flux, developed in Julia, and PyTorch, which operates on Python and C++. We executed five identical, consecutive trials for each framework. The benchmark utilized a standard convolutional neural network architecture trained on the complete FashionMNIST dataset. Across all experiments, the learning rate was set to 0.01, the batch size to 10, and the models were trained for 3 epochs utilizing a standard gradient descent optimizer. Conducting five consecutive runs allows us to isolate the initial startup overhead from the actual steady-state training speed.

TABLE I
OUR NETWORK EFFICIENCY RESULTS

Trial	Time (s)	Allocs	Mem (GB)	Test Acc (%)
1	21.74	$4,24 \times 10^7$	2.47	86.53
2	19.13	$3,22 \times 10^7$	2.03	86.76
3	19.86	$3,22 \times 10^7$	2.03	86.44
4	19.49	$3,22 \times 10^7$	2.03	85.42
5	18.97	$3,22 \times 10^7$	2.03	86.45

TABLE II
FLUX EFFICIENCY RESULTS

Trial	Time (s)	Allocs	Mem (GB)	Test Acc (%)
1	42.47	$6,01 \times 10^7$	26.09	86.88
2	29.12	$1,25 \times 10^7$	23.83	86.65
3	28.96	$1,25 \times 10^7$	23.83	86.50
4	26.76	$1,25 \times 10^7$	23.83	85.98
5	27.62	$1,25 \times 10^7$	23.83	86.52

The results for our custom library and Flux indicate that the first trial requires significantly more execution time and memory. This behavior is a direct consequence of Julia’s Just-In-Time (JIT) compilation model. During the initial run, the compiler must translate the code, infer variable types, and generate the final machine instructions. However, it is important to note that our custom engine experiences noticeably less compilation overhead during this first run compared to Flux.

The subsequent four trials are significantly faster because they utilize the previously compiled code. PyTorch does not exhibit this initial delay, as its core functions are pre-compiled in C++, resulting in highly consistent execution times across all five tests.

TABLE III
PYTORCH EFFICIENCY RESULTS

Trial	Time (s)	Test Acc (%)
1	32.94	85.75
2	32.52	86.79
3	32.47	86.01
4	32.21	86.00
5	33.09	86.66

Memory management was a primary focus during the development process. Comparing our library to Flux reveals a substantial reduction in memory consumption. While Flux required approximately 23.83 GB of memory during a steady-state run, our engine consumed only 2.03 GB, which is less than 10 percent of Flux’s footprint. It should be noted that memory allocation data is omitted for PyTorch. We utilized Julia’s native profiling tools to measure memory usage; however, because PyTorch manages its memory externally via C++, it cannot be evaluated equitably using the same methodology.

TABLE IV
AVERAGE EFFICIENCY RESULTS (5 TRIALS)

Library	Time (s)	Train Acc (%)	Test Acc (%)
Our library	19.84	87.57	86.32
Flux	30.99	87.73	86.51
PyTorch	32.65	87.47	86.24

Analyzing the average results across all five trials, our custom library demonstrated the fastest execution. It achieved an average time of 19.84 seconds, compared to 30.99 seconds for Flux and 32.65 seconds for PyTorch. This confirms that the applied code optimizations effectively accelerated the training process.

Furthermore, all three libraries reached nearly identical accuracy levels. Since the exact same network architecture was used for every test, comparable final results were expected. The training accuracy stabilized around 87.5 percent, while the test accuracy reached approximately 86.3 percent. Obtaining identical results to established standard libraries under the same conditions serves as clear validation that our mathematical implementations and gradient calculations are correct.

V. CONCLUSION

In this article, we presented a custom, highly optimized Automatic Differentiation engine in Julia that successfully competes with established frameworks. By eliminating type instability and utilizing in-place operations, our engine achieved the fastest average training time of 19.84 seconds and reduced memory consumption to less than 10% of what Flux requires, while maintaining identical mathematical accuracy.

Despite these excellent results, the library has areas for potential improvement. Minor hidden memory allocations in

layers like Dropout and Softmax could be eliminated by pre-allocating memory buffers, and the execution of heavy mathematical operations could be further optimized at the compiler level. Furthermore, the efficiency of convolutional layers could be enhanced by implementing the convolution operation using the Fast Fourier Transform (FFT) as a faster alternative to the current *im2col* algorithm. Future development should also focus on integrating GPU acceleration, adopting compile-time gradient calculations to bypass initial JIT compilation delays, and expanding support for more complex architectures with more layers.

CODE AVAILABILITY

The complete source code developed for this study, including the custom AD engine and all benchmark scripts, is publicly available in our GitHub repository: https://github.com/PyIkoR/CNN_architecture_comparison.

REFERENCES

- [1] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic Differentiation in Machine Learning: a Survey," *Journal of Machine Learning Research*, vol. 18, no. 153, pp. 1–43, 2018.
- [2] J. Revels, M. Lubin, and T. Papamarkou, "Forward-Mode Automatic Differentiation in Julia," *arXiv preprint arXiv:1607.07892*, 2016.
- [3] K. Chellapilla, S. Puri, and P. Simard, "High Performance Convolutional Neural Networks for Document Processing," in *Tenth International Workshop on Frontiers in Handwriting Recognition*, Suisun City, CA, USA, Oct. 2006.
- [4] A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, pp. 8026–8037, 2019.
- [5] M. Innes, "Don't Unroll Adjoint: Differentiating SSA-Form Programs," *arXiv preprint arXiv:1810.07951*, 2018.
- [6] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, "Julia: A Fresh Approach to Numerical Computing," *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017.
- [7] W. S. Moses and V. Churavy, "Instead of Rewriting Foreign Code for Machine Learning, Automatically Synthesize Fast Gradients," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 12472–12485, 2020.